

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ



پروژه پایانی مقطع کارشناسی مهندسی نرم افزار کامپیوتر

دانشگاه: دانشگاه ملی مهارت - دانشکده شهید مہاجر اصفہان

گروه: کامپیوتر نرم افزار

موضوع پروژه: سامانه جامع رزرو و مدیریت حمل و نقل بین شهری

استاد داور: میثم شریفی

استاد راهنما: نوید ایمانی

دانشجو: علی انصاریپور

شماره دانشجویی: 02120022705011

کد ملی: 1190331608

مرداد 1405

فهرست مطالب

7	فصل اول: کلیات پروژه.....
8	1.1- معرفی پروژه.....
8	1.2- هدف پروژه.....
8	1.3- مسئله و نیازمندی‌های پروژه.....
8	1.4- کاربران و نقش‌های سیستم.....
9	1.5- قابلیت‌های اصلی سیستم.....
10	فصل دوم: فناوری‌ها و ابزارهای استفاده‌شده.....
11	2.1- زبان برنامه‌نویسی PHP.....
11	2.2- چارچوب Laravel.....
11	2.3- Livewire.....
11	2.4- Blade.....
11	2.5- Tailwind CSS.....
11	2.6- پایگاه‌داده MySQL.....
12	2.7- کتابخانه‌ها و Package های مهم.....
12	2.8- ابزارهای توسعه و Build.....
13	فصل سوم: معماری و ساختار برنامه.....
14	3.1- معماری کلی سیستم.....
14	3.2- ساختار پوشه‌ها.....
17	3.3- Model ها.....
17	3.4- Service ها و لایه منطق برنامه.....
18	3.5- Livewire Component ها.....
18	3.6- Blade Component ها و View ها.....
19	3.7- Trait ها و کلاس‌های پایه.....
19	3.8- Enum ها.....
19	3.9- Middleware و Authorization.....
20	فصل چهارم: ساختار پایگاه‌داده.....
21	4.1- معرفی پایگاه‌داده.....

21	Schema	4.2
22	روابط بین جداول	4.3
23	جدول users	4.4
23	جدول terminals	4.5
24	جدول transit_lines	4.6
24	جدول transit_services	4.7
24	جدول transit_service_price_histories	4.8
25	جدول سلسله مراتب مناطق ایران	4.9
25	جدول provinces	4.9.1
25	جدول counties	4.9.2
25	جدول districts	4.9.3
26	جدول settlements	4.9.4
26	جدول villages	4.9.5
27	فصل پنجم: منطق و عملکرد برنامه	
28	مدیریت پایانه ها	5.1
28	هدف و ورودی ها	5.1.1
28	مراحل پردازش	5.1.2
28	فایل ها و کلاس های درگیر	5.1.3
29	خروجی و نکات فنی	5.1.4
29	واسط کاربری برنامه	5.1.5
31	مدیریت خطوط تردد	5.2
31	هدف و ورودی ها	5.2.1
31	مراحل پردازش	5.2.2
31	فایل ها و کلاس های درگیر	5.2.3
31	خروجی و نکات فنی	5.2.4
32	واسط کاربری برنامه	5.2.5
35	مدیریت سرویس های تردد	5.3
35	هدف و ورودی ها	5.3.1
35	مراحل پردازش	5.3.2

35	5.3.3- فایل ها و کلاس های درگیر
35	5.3.4- خروجی و نکات فنی
35	5.1.5- واسط کاربری برنامه
35	5.4- شبیه ساز اشغالی - Occupancy Simulator
35	5.4.1- هدف و ورودی ها
35	5.4.2- مراحل پردازش
35	5.4.3- فایل ها و کلاس های درگیر
35	5.4.4- خروجی و نکات فنی
35	5.5- آبخاری انتخاب مناطق جغرافیایی
35	5.5.1- هدف و ورودی ها
35	5.5.2- مراحل پردازش
36	5.5.3- فایل ها و کلاس های درگیر
36	5.5.4- خروجی و نکات فنی
37	فصل ششم: موتور قیمت گذاری پویا
38	6.1- هدف قابلیت
38	6.2- ورودی ها
38	6.3- مراحل پردازش
39	6.4- منطق اصلی
39	6.5- فایل ها و کلاس های درگیر
40	6.6- تعامل با پایگاه داده
40	6.7- خروجی
40	6.8- نکات فنی مهم
41	فصل هفتم: بررسی فایل های مهم پروژه
42	7.1- لایه Service
42	7.2- لایه Model
43	7.3- لایه Livewire و View
44	7.4- لایه پیکربندی و Route
45	فصل هشتم: جریان های اصلی سیستم
46	8.1- جریان محاسبه مجدد قیمت پویا

46	8.2- جریان حذف با محافظت و مودال
47	8.3- جریان آبخاری انتخاب منطقه
47	8.4- جریان نمایش داده‌های جدول (Datatable)
48	نگاهی به آینده
49	منابع

فصل اول: کلیات پروژه

1.1- معرفی پروژه

TransitApp (با نام تجاری داخلی پایانه‌یار) یک سامانه تحت وب به زبان فارسی و با چیدمان راست‌به‌چپ (RTL) است که جهت مدیریت پایانه‌های حمل‌ونقل، خطوط تردد و سرویس‌های حرکتی توسعه یافته است. این پروژه با چارچوب Laravel و به صورت یک پنل مدیریتی پیاده‌سازی شده و تمامی عملیات ثبت، ویرایش، حذف و جستجو را برای سه موجودیت اصلی — پایانه، خط تردد و سرویس تردد — پوشش می‌دهد.

سامانه از یک موتور قیمت‌گذاری پویا بهره می‌برد که قیمت هر سرویس را بر اساس فاصله، مدت زمان، زمان باقی‌مانده تا حرکت، میزان اشغال صندلی و وضعیت VIP به صورت خودکار محاسبه و یک تاریخچه کامل از هر محاسبه را در پایگاه داده ثبت می‌کند.

1.2- هدف پروژه

هدف اصلی این پروژه، ایجاد یک زیرساخت نرم‌افزاری یکپارچه برای مدیریت اطلاعات پایانه‌ها و سرویس‌های بین‌شهری است. با توجه به پیچیدگی‌های محاسباتی قیمت‌گذاری در صنعت حمل‌ونقل، تمرکز ویژه‌ای بر طراحی یک موتور قیمت‌گذاری انعطاف‌پذیر و قابل حسابرسی قرار گرفته است. این سامانه می‌تواند به عنوان هسته اولیه یک سیستم رزرواسیون یا مدیریت ناوگان مورد استفاده قرار گیرد.

1.3- مسئله و نیازمندی‌های پروژه

مسئله اصلی که این پروژه به دنبال حل آن است، عبارت است از:

- مدیریت مکانی پایانه‌ها: پایانه‌ها باید به سلسله‌مراتب تقسیمات کشوری (استان ← شهرستان ← بخش ← آبادی ← روستا) متصل باشند.
- مدیریت خطوط تردد: هر خط تردد دارای مبدأ و مقصد، قیمت پایه، مسافت تخمینی و مدت زمان تخمینی است.
- مدیریت سرویس‌های تردد: هر سرویس یک حرکت واقعی در یک خط تردد است که دارای زمان حرکت، زمان رسیدن، ظرفیت و وضعیت اشغال می‌باشد.
- قیمت‌گذاری پویا: قیمت نهایی هر سرویس نباید ثابت باشد؛ بلکه باید بر اساس پارامترهای لحظه‌ای و مشخصه‌های خط محاسبه شود.
- حفظ تاریخچه قیمت: برای شفافیت و حسابرسی، هر بار که قیمت یک سرویس محاسبه می‌شود، علاوه بر ذخیره قیمت جدید، جزئیات کامل محاسبه (ضرایب، قیمت قبلی و درصد تغییر) باید ثبت گردد.

1.4- کاربران و نقش‌های سیستم

در پروژه یک Enum برای نقش‌های کاربری تعریف شده است که مقادیر terminal, admin, user و company را شامل می‌شود. این نقش‌ها در جدول users در قالب یک فیلد role ذخیره می‌شوند. با این حال، در نسخه فعلی پروژه هیچ سامانه احراز هویت (Authentication)، لاگین، ثبت‌نام یا میان‌افزار (Middleware) کنترل دسترسی بر اساس نقش پیاده‌سازی نشده است و تمامی صفحات به صورت عمومی در دسترس می‌باشند.

1.5- قابلیت‌های اصلی سیستم

قابلیت‌های اصلی سامانه در نسخه فعلی عبارتند از:

- ثبت، ویرایش، حذف و مشاهده پایانه‌ها با انتخاب آبشاری مناطق جغرافیایی ایران
- ثبت، ویرایش، حذف و مشاهده خطوط تردد بین پایانه‌ها
- ثبت، ویرایش، حذف و مشاهده سرویس‌های تردد متعلق به هر خط
- محاسبه خودکار قیمت سرویس‌ها بر اساس شش عامل متغیر
- نگهداری تاریخچه کامل محاسبات قیمت برای هر سرویس
- جستجو، مرتب‌سازی، فیلتر و انتخاب گروهی رکوردها در تمام جداول
- شبیه‌ساز خودکار اشغالی جهت نمایش رفتار موتور قیمت‌گذاری

فصل دوم: فناوری‌ها و ابزارهای استفاده‌شده

2.1- زبان برنامه‌نویسی PHP

پروژه TransitApp با زبان PHP نسخه 8.4 توسعه یافته است. PHP یک زبان اسکریپتی سمت سرور است که به‌طور گسترده در توسعه برنامه‌های وب استفاده می‌شود و پایه اصلی چارچوب Laravel را تشکیل می‌دهد.

2.2- چارچوب Laravel

Laravel یک چارچوب متن‌باز و رایگان برای توسعه برنامه‌های وب با زبان PHP است که بر پایه معماری MVC طراحی شده است. در این پروژه از نسخه 13.0^ Laravel استفاده شده است. Laravel امکاناتی مانند ORM قدرتمند (Eloquent)، سیستم Migration، مسیریابی انعطاف‌پذیر، سیستم صف و ابزار خط فرمان Artisan را در اختیار توسعه‌دهنده قرار می‌دهد.

2.3- Livewire

Livewire یک چارچوب full-page component برای Laravel است که امکان ساخت رابط‌های کاربری پویا و واکنش‌گرا را بدون نیاز به نوشتن کدهای جاوااسکریپت پیچیده فراهم می‌کند. در این پروژه از نسخه 4.1 Livewire استفاده شده و تمامی صفحات پروژه به‌صورت Componentهای Livewire full-page پیاده‌سازی شده‌اند. این بدان معناست که هیچ Controller عملیاتی در پروژه وجود ندارد و تمام منطق نمایش و تعامل درون Componentهای Livewire متمرکز شده است.

2.4- Blade

Blade موتور قالب‌سازی Laravel است. در این پروژه، Blade برای طراحی Layoutهای اصلی، Componentهای UI و قالب‌های همراه Componentهای Livewire استفاده شده است. فایل‌های Blade با پسوند blade.php در مسیر resources/views قرار دارند و از قابلیت‌هایی مانند ارث‌بری Layout، Componentها و دستورات شرطی و حلقه بهره می‌برند.

2.5- Tailwind CSS

Tailwind CSS یک چارچوب CSS مبتنی بر Utility Class است. در این پروژه از نسخه 4.1.12 Tailwind CSS استفاده شده است. تنظیمات تم در فایل resources/css/app.css تعریف شده و فونت IRANSans برای نمایش فارسی در پروژه به کار رفته است. تمام صفحات با چیدمان راست‌به‌چپ (RTL) طراحی شده‌اند.

2.6- پایگاه‌داده MySQL

پایگاه‌داده پروژه MySQL است که از طریق فایل env با مقادیر DB_DATABASE=transit_app و DB_CONNECTION=mysql پیکربندی شده است. MySQL یک سیستم مدیریت پایگاه‌داده رابطه‌ای متن‌باز است که برای ذخیره‌سازی تمام داده‌های سامانه، از جمله اطلاعات پایانه‌ها، خطوط، سرویس‌ها و سلسله‌مراتب مناطق ایران استفاده می‌شود.

2.7- کتابخانه‌ها و Package های مهم

Package های اصلی استفاده شده در پروژه عبارتند از:

کاربرد	نسخه	Package
چارچوب اصلی Laravel	^13.0	laravel/framework
Component های full-page و reactive	^4.1	livewire/livewire
مودال های تأیید حذف	^3.0	wire-elements/modal
مدیریت تاریخ های شمسی (Jalali)	^3.5	morilog/jalali
محیط تعاملی REPL	^3.0	laravel/tinker

Package های فرانت اند اصلی:

کاربرد	نسخه	Package
استایل دهی UI	^4.1.12	tailwindcss
رفتارهای جاوااسکریپت سبک	^3.14.9	alpinejs
انیمیشن باز/بسته شدن	^3.15.12	@alpinejs/collapse
آیکون ها	^7	@fortawesome/fontawesome-free

2.8- ابزارهای توسعه و Build

فرآیند Build پروژه با Vite نسخه 7 و Plugin Laravel انجام می شود. دستور `npm run build` فایل های CSS و JS را برای محیط Production فشرده سازی می کند. در محیط Development از دستور `npm run dev` استفاده می شود. همچنین ابزارهای توسعه Laravel مانند Pint (فرمت کننده کد)، Sail (Docker) و PHPUnit برای تست در پروژه موجود هستند، هرچند در نسخه فعلی تست کاربردی نوشته نشده است.

فصل سوم: معماری و ساختار برنامه

3.1- معماری کلی سیستم

معماری پروژه TransitApp بر پایه الگوی Livewire v4 Full-Page Components به همراه لایه Service طراحی شده است. برخلاف الگوی کلاسیک MVC که در آن Controllerها وظیفه دریافت درخواست و ارسال پاسخ را بر عهده دارند، در این پروژه هیچ Controller عملیاتی وجود ندارد و تمامی مسیرها (Routes) مستقیماً Componentهای Livewire را رندر می کنند.

لایه های اصلی سامانه:

1. لایه نمایش (View): قالب های Blade و Componentهای Livewire
2. لایه منطق (Service): کلاس های Service که عملیات CRUD، فیلتر، جستجو و قوانین حذف را مدیریت می کنند
3. لایه داده (Model): مدل های Eloquent و Migrationها
4. لایه ترکیب رفتار Trait: Traitهایی مانند withDatatable، withModal و withRegionSelects که رفتارهای مشترک صفحات را فراهم می کنند

3.2- ساختار پوشه ها

ساختار اصلی پوشه های پروژه به شرح زیر است:

- **TransitApp**
 - ریشه اصلی پروژه است و تمامی فایل ها و پوشه های مربوط به سامانه TransitApp در آن قرار دارند.
 - **app**
 - شامل منطق اصلی برنامه، کلاس ها، مدل ها، سرویس ها و سایر اجزای سمت سرور است.
 - **Class**
 - شامل کلاس های پایه و کلاس های عمومی مورد استفاده در بخش های مختلف برنامه است.
 - شامل Service.php برای تعریف ساختار پایه سرویس ها و ModalComponent.php برای مدیریت ساختار کامپوننت های مودال است.
 - **Enums**
 - شامل Enum های نوع Backed است که برای تعریف مقادیر ثابت و محدود در بخش های مختلف سامانه استفاده می شوند.
 - از جمله:
 - UserRoleEnum
 - SettlementTypeEnum
 - و سایر Enum های مورد نیاز سامانه.
 - **Exceptions**
 - شامل Exception های اختصاصی برنامه است.
 - CannotDeleteException برای مدیریت شرایطی استفاده می شود که حذف یک رکورد امکان پذیر نیست.

- **Helpers**
 - شامل توابع و کلاس‌های کمکی مورد استفاده در بخش‌های مختلف برنامه است.
 - translation.php برای توابع مرتبط با ترجمه.
 - MenuHelper.php برای مدیریت و تولید منوها.
 - ModelHelper.php برای عملیات و قابلیت‌های کمکی مرتبط با مدل‌ها.
- **Http/Controllers**
 - محل قرارگیری Controller های برنامه است.
 - در این پروژه تنها Controller.php وجود دارد که یک کلاس abstract و پایه برای Controller ها است.
- **Livewire**
 - شامل Component های Livewire مرتبط با رابط کاربری پویا است.
 - در این پروژه عمدتاً برای Component های مربوط به مودال حذف استفاده می‌شود.
- **Models**
 - شامل مدل‌های Eloquent است که ارتباط بین منطق برنامه و جداول پایگاه داده را برقرار می‌کنند.
- **IranRegion**
 - شامل مدل‌های مربوط به ساختار سلسله‌مراتبی مناطق جغرافیایی ایران است.
 - این ساختار برای مدیریت و ارتباط بین استان‌ها، شهرستان‌ها، بخش‌ها، سکونتگاه‌ها و روستاها استفاده می‌شود.
- **Observers**
 - شامل Observer های مربوط به مدل‌های Eloquent است.
 - TransitLineObserver برای اجرای منطق مورد نیاز هنگام رخ دادن رویدادهای مرتبط با مدل TransitLine استفاده می‌شود.
- **Providers**
 - شامل Service Provider های برنامه است.
 - AppServiceProvider برای ثبت و پیکربندی سرویس‌های عمومی برنامه استفاده می‌شود.
- **Services**
 - شامل لایه منطق اصلی برنامه است.
 - منطق پیچیده‌تر و مستقل از Controller ها در این بخش قرار می‌گیرد تا مسئولیت‌ها از یکدیگر تفکیک شوند.
- **Demo**
 - شامل سرویس‌های مورد استفاده برای قابلیت‌های نمایشی و شبیه‌سازی است.
 - OccupancySimulatorService برای شبیه‌سازی میزان اشغال سرویس‌های حمل‌ونقل استفاده می‌شود.
- **Traits**
 - شامل Trait های رفتاری است که قابلیت‌های مشترک را میان چند کلاس یا مدل قابل استفاده می‌کنند.

▪ View/Components

- شامل Component های Blade است که برای ایجاد اجزای قابل استفاده مجدد در رابط کاربری استفاده می‌شوند.

○ database

- شامل تمام فایل‌های مرتبط با ساخت و مقداردهی پایگاه داده است.

▪ factories

- شامل Factory های مورد استفاده برای تولید داده‌های آزمایشی و نمایشی است.

▪ migrations

- شامل Migration های پایگاه داده است.

- ساختار جداول، ستون‌ها، کلیدها و سایر تغییرات ساختاری پایگاه داده از طریق این فایل‌ها مدیریت می‌شود.

▪ seeders

- شامل Seeder های مورد استفاده برای مقداردهی اولیه پایگاه داده است.

- از جمله داده‌های مربوط به:

- مناطق ایران.

- پایانه‌ها.

- کاربران.

- و سایر داده‌های اولیه مورد نیاز سامانه.

○ lang/fa

- شامل فایل‌های ترجمه فارسی سامانه است.

- متن‌های قابل ترجمه و پیام‌های فارسی برنامه در این بخش نگهداری می‌شوند.

○ resources/views

- شامل View های Blade و فایل‌های مربوط به لایه نمایش سامانه است.

▪ pages

- شامل Component های Full-Page Livewire است.

- هر Component می‌تواند یک صفحه کامل از سامانه را مدیریت کند.

▪ layouts

- شامل Layout های اصلی سامانه است.

- برای نمونه، app.blade.php ساختار اصلی صفحات برنامه را مشخص می‌کند.

▪ components

- شامل Component های Blade قابل استفاده مجدد است.

- این Component ها برای جلوگیری از تکرار کد در رابط کاربری استفاده می‌شوند.

▪ livewire

- شامل View های مربوط به Component های Livewire است.

- در این پروژه، View های مربوط به مودال‌های حذف در این بخش قرار دارند.

- شامل فایل‌های تعریف مسیرهای سامانه است.
- **web.php**
- شامل ۱۲ مسیر مربوط به صفحات و Component های Livewire است.
- مسیرهای اصلی رابط وب سامانه در این فایل تعریف شده‌اند.
- **console.php**
- شامل تنظیمات مربوط به دستورات و زمان‌بندی‌های کنسولی است.
- زمان‌بندی اجرای شبیه‌ساز میزان اشغال سرویس‌ها در این بخش انجام می‌شود.

3.3- Model ها

مدل‌های Eloquent پروژه در مسیر `app/Models` قرار دارند و عبارتند از:

- `User`: مدل کاربران با `cast` نقش به `UserRoleEnum`
- `Terminal`: پایانه با روابط به سلسله‌مراتب مناطق و خطوط تردد
- `TransitLine`: خط تردد با روابط مبدأ و مقصد (پایانه) و سرویس‌ها
- `TransitService`: سرویس تردد با روابط به خط و تاریخچه قیمت‌ها
- `TransitServicePriceHistory`: تاریخچه محاسبات قیمت
- `Village, Settlement, District, County, Province`: مدل‌های سلسله‌مراتب مناطق ایران

3.4- Service ها و لایه منطق برنامه

لایه Service مرکزی‌ترین بخش منطق برنامه است. کلاس پایه `App\Class\Service` عملیات CRUD، صفحه‌بندی، جستجو و

مرتب‌سازی را برای هر مدل Eloquent فراهم می‌کند. سرویس‌های مشتق شده شامل:

- `TerminalService`: CRUD پایانه‌ها + فیلتر منطقه‌ای + محافظت حذف
- `TransitLineService`: CRUD خطوط + فیلتر قیمت/مبدأ/مقصد + محافظت حذف
- `TransitServiceService`: CRUD سرویس‌ها + eager loading
- `TransitPricingService`: موتور محاسبه قیمت پویا
- `RegionsService`: فراخوانی گزینه‌های آبخاری مناطق
- `OccupancySimulatorService`: شبیه‌ساز لحظه‌ای اشغالی (دمو)

3.5 Livewire Component ها

تمامی صفحات پروژه Component های Livewire هستند که از طریق `Route::livewire()` در `routes/web.php` تعریف شده‌اند. این Component ها به دو صورت پیاده‌سازی شده‌اند:

1. Single-File Components (SFC): کلاس‌های ناشناس (anonymous) درون فایل `blade.php` با پیشوند ⚡
 2. Multi-File Components: پوشه‌های ⚡ `<name>/` حاوی فایل‌های `index.php` و `index.blade.php`
- Component های صفحه عبارتند از:

- `home.blade.php` — صفحه اصلی ⚡
- `dashboard.blade.php` — داشبورد مدیریتی ⚡
- `/terminals/` — فهرست پایانه‌ها ⚡
- `/terminals/` — فرم ایجاد/ویرایش پایانه ⚡
- `/transit-lines/` — فهرست خطوط تردد ⚡
- `/transit-lines/` — فرم ایجاد/ویرایش خط ⚡
- `/transit-services/` — فهرست سرویس‌ها ⚡
- `/transit-services/` — فرم ایجاد/ویرایش سرویس ⚡

همچنین Component های کلاس‌محور (Class-based) مودال حذف در مسیر `app/Livewire/.../Modals/Delete.php` قرار دارند.

3.6 Blade Component ها و View ها

Component های Blade در مسیر `resources/views/components` دسته‌بندی شده‌اند و شامل موارد زیر هستند:

- Component — `common/` — المان‌های مشترک
- `/forms` — المان‌های فرم
- `/header` — هدرها
- Layout — `layouts/` — المان‌های فرعی
- Component — `tables/` — المان‌های جدول (سرستون قابل مرتب‌سازی، صفحه‌بندی و ...)
- `/ui` — المان‌های رابط کاربری

Layout اصلی برنامه `resources/views/layouts/app.blade.php` است که `<html dir="rtl" lang="fa">` را تنظیم می‌کند و فونت IRANSans را بارگذاری می‌نماید.

3.7-Trait ها و کلاس‌های پایه

Trait های اصلی پروژه عبارتند از:

- withDatatable: جستجو، مرتب‌سازی، صفحه‌بندی، انتخاب گروهی و حفظ وضعیت در URL
- withModal: باز کردن مودال‌های wire-elements/modal
- withRegionSelects: Dropdown های آبخاری استان/شهرستان/بخش/آبادی/روستا
- withTableDelete: حذف تکروردی و گروهی با مدیریت خطا
- withToastNotification: نمایش اعلان‌های Toast
- EnumHelperTrait: متدهای کمکی برای Enum ها (label, values, fromValue)

کلاس‌های پایه:

- App: کلاس abstract پایه برای تمام سرویس‌ها
- App: کلاس پایه Component های مودال

3.8-Enum ها

Enum های تعریف شده در مسیر app/Enums:

- UserRoleEnum: نقش‌های user, admin, terminal, company
- SettlementTypeEnum: انواع آبادی (city, urban_area, rural_district)
- TransitServiceVehicleType: تعریف شده اما در Migration ها استفاده نشده است
- VillageTypeEnum: تعریف شده اما در هیچ Model یا Migration ارجاع داده نشده است

3.9- Authorization و Middleware

در نسخه فعلی پروژه هیچ Middleware سفارشی، Policy یا Gate تعریف نشده است. همچنین سیستم احراز هویت (Authentication) و کنترل دسترسی بر اساس نقش (Authorization) پیاده‌سازی نشده است. فیلد role در جدول users صرفاً به صورت یک فیلد داده‌ای ذخیره می‌شود و هیچ محدودیتی بر اساس آن اعمال نمی‌گردد.

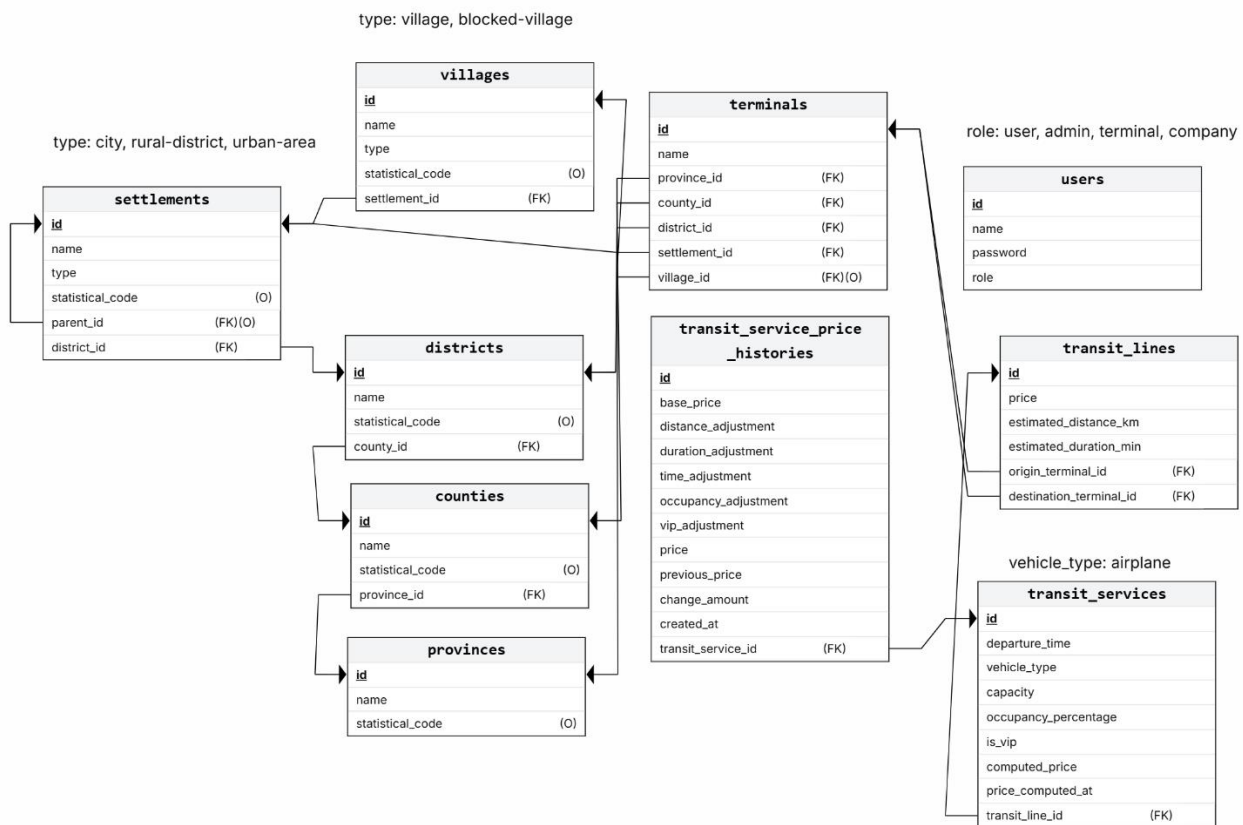
فصل چهارم: ساختار پایگاه داده

4.1- معرفی پایگاه داده

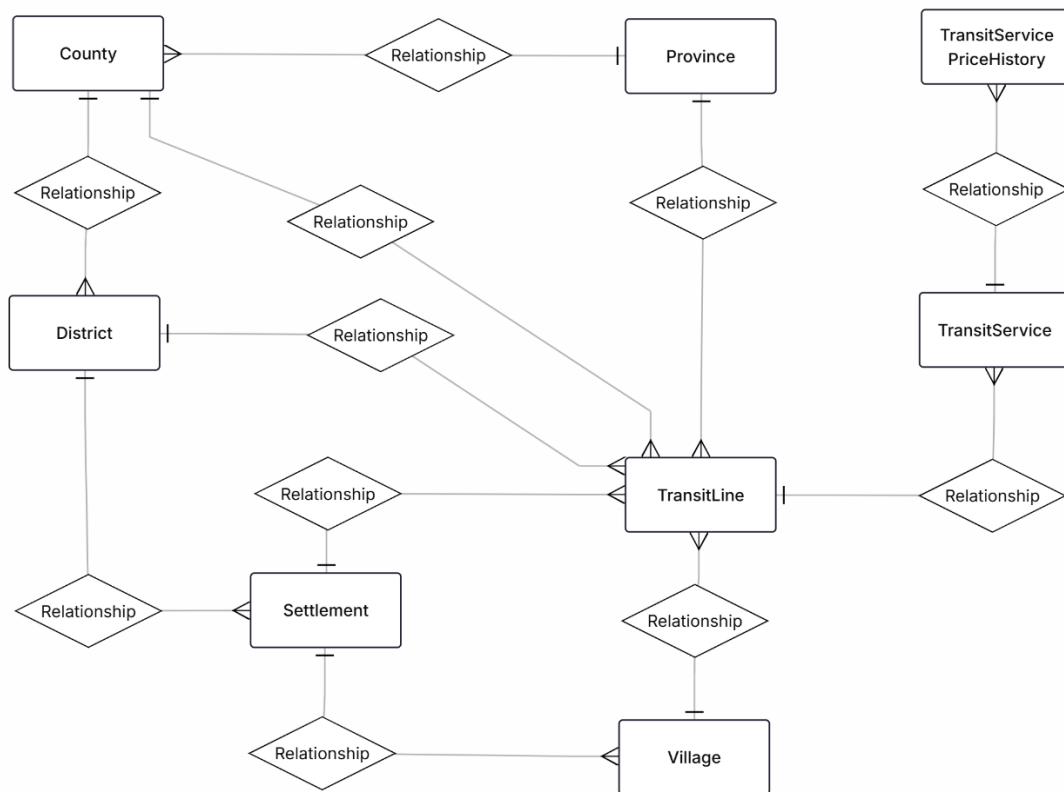
پایگاه داده پروژه MySQL نام transit_app است و شامل 17 جدول می باشد. این جداول در 6 فایل Migration ایجاد شده اند. جداول به سه دسته تقسیم می شوند:

1. جداول failed_jobs, job_batches, jobs, cache_locks, cache, sessions, Laravel: users جداول
2. جداول مناطق ایران: provinces, counties, districts, settlements, villages جداول
3. جداول دامنه کسب و کار: terminals, transit_lines, transit_services, transit_service_price_histories جداول

4.2- Schema



4.3- روابط بین جداول



روابط اصلی پایگاه داده به شرح زیر است:

- `villages, settlements, districts, counties, provinces → terminals` (چند به یک) \
- `transit_lines → terminals` (مبدأ و مقصد، دو رابطه چند به یک)
- `transit_services → transit_lines` (چند به یک)
- `transit_service_price_histories → transit_services` (چند به یک)
- `counties → provinces` (چند به یک)
- `districts → counties` (چند به یک)
- `settlements → districts` (چند به یک)
- `villages → settlements` (چند به یک)

4.4- جدول users

این جدول اطلاعات کاربران را ذخیره می‌کند.

ستون	نوع	توضیحات
id	BIGINT UNSIGNED auto increment	شناسه یکتای رکورد — کلید اصلی
name	VARCHAR(255)	نام کاربر
email	VARCHAR(255) unique	آدرس ایمیل
email_verified_at	TIMESTAMP nullable	زمان تأیید ایمیل
password	VARCHAR(255)	رمز عبور
role	ENUM('user','admin','terminal','company')	نقش کاربر (پیش‌فرض user):
remember_token	VARCHAR(100) nullable	توکن "مرا به خاطر بسپار"
created_at / updated_at	TIMESTAMP	زمان ایجاد و آخرین ویرایش

4.5- جدول terminals

این جدول اطلاعات پایانه‌های حمل‌ونقل را ذخیره می‌کند.

ستون	نوع	توضیحات
id	BIGINT UNSIGNED auto increment	شناسه یکتا — کلید اصلی
name	VARCHAR(255)	نام پایانه
phone	VARCHAR(20) nullable	شماره تماس
code	VARCHAR(50) nullable	کد پایانه
province_id	BIGINT UNSIGNED	کلید خارجی provinces →
county_id	BIGINT UNSIGNED	کلید خارجی counties →
district_id	BIGINT UNSIGNED	کلید خارجی districts →
settlement_id	BIGINT UNSIGNED	کلید خارجی settlements →
village_id	BIGINT UNSIGNED nullable	کلید خارجی villages → اختیاری
created_at / updated_at	TIMESTAMP	زمان ایجاد و ویرایش

4.6- جدول transit_lines

این جدول خطوط تردد بین پایانه‌ها را تعریف می‌کند.

ستون	نوع	توضیحات
id	BIGINT UNSIGNED auto increment	شناسه یکتا — کلید اصلی
origin_terminal_id	BIGINT UNSIGNED	کلید خارجی (terminals → مبدأ)
destination_terminal_id	BIGINT UNSIGNED	کلید خارجی (terminals → مقصد)
base_price	BIGINT UNSIGNED default 0	قیمت پایه خط
estimated_distance_km	DECIMAL(8,2) nullable	مسافت تخمینی (کیلومتر)
estimated_duration_min	MEDIUMINT UNSIGNED nullable	مدت زمان تخمینی (دقیقه)
created_at / updated_at	TIMESTAMP	زمان ایجاد و ویرایش

4.7- جدول transit_services

این جدول سرویس‌های تردد (حرکت‌های واقعی) را ذخیره می‌کند.

ستون	نوع	توضیحات
id	BIGINT UNSIGNED auto increment	شناسه یکتا — کلید اصلی
transit_line_id	BIGINT UNSIGNED	کلید خارجی transit_lines → (حذف آشناری)
name	VARCHAR(255) nullable	نام سرویس (اختیاری)
base_price	BIGINT UNSIGNED default 0	قیمت پایه (کپی از خط)
computed_price	BIGINT	قیمت محاسبه‌شده زنده
departure_time	TIME nullable	زمان حرکت
arrival_time	TIME nullable	زمان رسیدن
occupancy	FLOAT default 0	نسبت اشغال (بازه [0,1])
total_seats	SMALLINT UNSIGNED default 40	تعداد کل صندلی‌ها
is_vip	BOOLEAN default false	وضعیت VIP
created_at / updated_at	TIMESTAMP	زمان ایجاد و ویرایش

4.8- جدول transit_service_price_histories

این جدول تاریخچه کامل محاسبات قیمت هر سرویس را نگهداری می‌کند.

ستون	نوع	توضیحات
id	BIGINT UNSIGNED auto increment	شناسه یکتا — کلید اصلی
transit_service_id	BIGINT UNSIGNED	کلید خارجی transit_services → (حذف آشناری)

تاریخ سفر (میلادی ذخیره، شمسی نمایش)	DATE	trip_date
ضریب مدت زمان	DECIMAL nullable	duration_factor
ضریب زمان باقی مانده	DECIMAL nullable	time_factor
ضریب اشغال	DECIMAL nullable	occupancy_factor
ضریب مسافت پایه	DECIMAL nullable	base_distance_factor
ضریب دقت مسافت	DECIMAL nullable	distance_accuracy_factor
ضریب VIP	DECIMAL nullable	vip_factor
قیمت محاسبه شده نهایی	BIGINT	price
قیمت قبلی	BIGINT nullable	previous_price
مقدار تغییر	BIGINT nullable	change_amount
درصد تغییر	DECIMAL nullable	change_percent
زمان ایجاد و ویرایش	TIMESTAMP	created_at / updated_at

4.9- جداول سلسله مراتب مناطق ایران

4.9.1- جدول provinces

ستون	نوع	توضیحات
id	BIGINT UNSIGNED auto increment	شناسه استان
code	unique	کد استان
name	VARCHAR	نام استان

4.9.2- جدول counties

ستون	نوع	توضیحات
id	BIGINT UNSIGNED auto increment	شناسه شهرستان
province_id	BIGINT UNSIGNED	کلید خارجی provinces →
code	unique	کد شهرستان
name	VARCHAR	نام شهرستان

4.9.3- جدول districts

ستون	نوع	توضیحات
id	BIGINT UNSIGNED auto increment	شناسه بخش
county_id	BIGINT UNSIGNED	کلید خارجی counties →
code	unique	کد بخش
name	VARCHAR	نام بخش

4.9.4- settlements جدول

ستون	نوع	توضیحات
id	BIGINT UNSIGNED auto increment	شناسه آبادی
district_id	BIGINT UNSIGNED	کلید خارجی districts →
type	ENUM('city','urban_area','rural_district')	نوع آبادی
code	–	کد آبادی

4.9.5- villages جدول

ستون	نوع	توضیحات
id	BIGINT UNSIGNED auto increment	شناسه روستا
settlement_id	BIGINT UNSIGNED	کلید خارجی settlements →
code	–	کد روستا
name	VARCHAR	نام روستا

فصل پنجم: منطق و عملکرد برنامه

5.1- مدیریت پایانه‌ها

5.1.1- هدف و ورودی‌ها

این قابلیت امکان ثبت، ویرایش، حذف و مشاهده پایانه‌های حمل‌ونقل را فراهم می‌کند. هر پایانه دارای نام، شماره تماس، کد و موقعیت جغرافیایی در سلسله‌مراتب مناطق ایران است.

ورودی‌ها:

- name (اجباری): نام پایانه
- phone (اختیاری): شماره تماس
- code (اختیاری): کد پایانه
- settlement_id, district_id, county_id, province_id (اجباری): سلسله‌مراتب منطقه‌ای
- village_id (اختیاری): روستا در صورت نیاز

5.1.2- مراحل پردازش

1. کاربر از صفحه داشبورد یا منوی کناری گزینه "پایانه‌ها" را انتخاب می‌کند.
2. Component terminals.index از Trait withDatatable برای بارگذاری داده‌ها استفاده می‌کند.
3. TerminalService::query () با اعمال جستجو، مرتب‌سازی و فیلترهای منطقه‌ای، داده‌ها را صفحه‌بندی شده باز می‌گرداند.
4. در فرم ایجاد/ویرایش، Trait withRegionSelects Dropdown های آبشاری استان تا روستا را مدیریت می‌کند.
5. هنگام حذف، TerminalService::delete () بررسی می‌کند که آیا پایانه در هیچ خط ترددی به‌عنوان مبدأ یا مقصد استفاده شده است یا خیر. در صورت استفاده، خطای CannotDeleteException پرتاب می‌شود.

5.1.3- فایل‌ها و کلاس‌های درگیر

- app/Services/TerminalService.php
- app/Models/Terminal.php
- resources/views/pages/terminals/⚡index/index.php
- resources/views/pages/terminals/⚡index/index.blade.php
- resources/views/pages/terminals/⚡form/form.php
- resources/views/pages/terminals/⚡form/form.blade.php
- app/Traits/withDatatable.php
- app/Traits/withRegionSelects.php

5.1.4- خروجی و نکات فنی

خروجی: فهرست صفحه‌بندی‌شده پایانه‌ها با قابلیت جستجو بر اساس نام و فیلتر بر اساس منطقه.
نکات فنی: - آدرس پایانه از طریق Accessor address () در Model Terminal ساخته می‌شود که نام آبادی، روستا و بخش را ترکیب می‌کند. - TerminalService فقط province را eager load می‌کند؛ بنابراین نمایش آدرس کامل در جدول می‌تواند منجر به N+1 query شود.

5.1.5- واسط کاربری برنامه

5.1.5.1- صفحه فهرست پایانه‌ها

The screenshot displays the 'پایانه‌ها' (Terminals) management page. At the top, there's a search bar and a filter dropdown set to 'ردیف' (Row). Below this is a table with the following data:

نام پایانه	آدرس	تاریخ ایجاد	عملیات
خیابان قهرمانیان	خراسان رضوی - تربت جام - نصرآباد - بالاچام - کلاته صوفی	۱۴۰۵/۰۵/۱۸ ۱۴:۳۲:۴۸	[Edit] [Delete]
خیابان علی‌زمانی	خراسان رضوی - مشهد - احمدآباد - پیوه زن - بازه حیدر	۱۴۰۵/۰۵/۱۸ ۱۴:۳۲:۴۸	[Edit] [Delete]
خیابان پایور	گیلان - لنگرود - مرکزی - لنگرود	۱۴۰۵/۰۵/۱۸ ۱۴:۳۲:۴۷	[Edit] [Delete]
خیابان حقیقی	کردستان - دیواندره - مارال - هزارکانیان	۱۴۰۵/۰۵/۱۸ ۱۴:۳۲:۴۷	[Edit] [Delete]
خیابان عقیلی	ایلام - ایوان - زرنه - زرنه - مزرعه گاومهر	۱۴۰۵/۰۵/۱۸ ۱۴:۳۲:۴۷	[Edit] [Delete]
خیابان کمالی	چهارمحال و بختیاری - بن - مرکزی - وردنجان - کرسنگ	۱۴۰۵/۰۵/۱۸ ۱۴:۳۲:۴۷	[Edit] [Delete]
خیابان فولادوند	اصفهان - اصفهان - مرکزی - قهجاورستان	۱۴۰۵/۰۵/۱۸ ۱۴:۳۲:۴۷	[Edit] [Delete]
خیابان ملک	گیلان - آستارا - مرکزی - آستارا	۱۴۰۵/۰۵/۱۸ ۱۴:۳۲:۴۷	[Edit] [Delete]
خیابان خولینی	گلستان - رامیان - مرکزی - دند	۱۴۰۵/۰۵/۱۸ ۱۴:۳۲:۴۷	[Edit] [Delete]
خیابان عصار	تهران - دماوند - مرکزی - دماوند	۱۴۰۵/۰۵/۱۸ ۱۴:۳۲:۴۷	[Edit] [Delete]

At the bottom of the table, there are pagination controls showing 'نمایش ۱ تا ۱۰ از ۲ نتیجه' (Display 1 to 10 of 2 results) and buttons for 'قبلی' (Previous), '۱', '۲', and 'بعدی' (Next).

فهرست حاوی پایانه‌ها با قابلیت صفحه‌بندی جهت افزایش سرعت نمایش ردیف‌ها + جستجو + مرتب‌سازی + ثبت جدید و ویرایش و حذف امن (بررسی وجود خطوط تردد وابسته به پایانه)

5.1.5.2- صفحه ساخت پایانه

پایانه یار

فهرست

خانه

مدیریت مسیرها

پایانه‌ها

خطوط تردد

سرویس‌های تردد

ایجاد پایانه

خانه > ایجاد پایانه

پایانه

نام پایانه

استان

انتخاب استان

بخش

انتخاب بخش

روستا (اختیاری)

انتخاب روستا

شهرستان

انتخاب شهرستان

دهستان/شهر

انتخاب دهستان/شهر

انصراف

ذخیره

ایجاد پایانه با راستی آزمایی هر مقدار و نمایش پیام های مناسب برای هر کاستی (ترجمه شده به فارسی)

5.1.5.3- صفحه ویرایش پایانه

پایانه یار

فهرست

مدیریت مسیرها

پایانه‌ها

خطوط تردد

سرویس‌های تردد

خانه

مدیریت مسیرها

پایانه‌ها

خطوط تردد

سرویس‌های تردد

پایانه

نام پایانه

خیابان قهرمانیان

استان

خراسان رضوی

بخش

نصرآباد

روستا (اختیاری)

کلاته صوفی

انصراف

ذخیره تغییرات

ویرایش پایانه با راستی آزمایی هر مقدار و نمایش پیام های مناسب برای هر کاستی (ترجمه شده به فارسی)

5.2- مدیریت خطوط تردد

5.2.1- هدف و ورودی‌ها

این قابلیت خطوط تردد بین دو پایانه را مدیریت می‌کند. هر خط دارای قیمت پایه، مسافت تخمینی و مدت زمان تخمینی است. ورودی‌ها:

- origin_terminal_id (اجباری): پایانه مبدأ
- destination_terminal_id (اجباری): پایانه مقصد
- base_price (اجباری): قیمت پایه (عدد صحیح ≥ 0)
- estimated_distance_km (اختیاری): مسافت تخمینی
- estimated_duration_min (اختیاری): مدت زمان تخمینی (دقیقه)

5.2.2- مراحل پردازش

1. کاربر پایانه مبدأ و مقصد را از Dropdown های بهینه‌شده انتخاب می‌کند.
2. در هنگام ذخیره خط جدید، Observer فعال نمی‌شود زیرا ستون‌های قیمت‌گذاری تغییر نکرده‌اند.
3. در هنگام ویرایش خط، اگر base_price، estimated_distance_km یا estimated_duration_min تغییر کند، TransitLineObserver::updated () تمام سرویس‌های متعلق به آن خط را مجدداً قیمت‌گذاری می‌کند.
4. حذف خط تنها در صورتی امکان‌پذیر است که هیچ سرویسی به آن متعلق نباشد.

5.2.3- فایل‌ها و کلاس‌های درگیر

- app/Services/TransitLineService.php
- app/Models/TransitLine.php
- app/Observers/TransitLineObserver.php
- resources/views/pages/transit-lines/⚡index/
- resources/views/pages/transit-lines/⚡form/

5.2.4- خروجی و نکات فنی

خروجی: فهرست خطوط با نمایش مبدأ، مقصد، قیمت پایه، مسافت و مدت زمان.
نکات فنی: - فیلتر بازه قیمت در TransitLineService::applyFilters () به ستون price ارجاع دارد که در Schema وجود ندارد (ستون صحیح base_price است). این امر موجب بروز خطای SQL هنگام استفاده از فیلتر قیمت می‌شود. - Observer قیمت‌گذاری مجدد ممکن است در صورت ویرایش سریع یا شرایط خاص، ریسک بازگشتی (recursion) داشته باشد.

5.2.5.1- صفحه فهرست خطوط تردد + فیلتر بر اساس مکان

فهرست حاوی خطوط تردد با قابلیت صفحه بندی جهت افزایش سرعت نمایش ردیف ها + جستجو + مرتب سازی + ثبت جدید و ویرایش و حذف امن (بررسی وجود سرویس های تردد وابسته به خط تردد) + فیلتر بر اساس تقسیمات کشوری و قیمت پایه

5.2.5.2- صفحه ساخت خط تردد

علی ✓

جستجو

پایانه یار

فهرست

خانه

مدیریت مسیرها

پایانه ها

خطوط تردد

سرویس های تردد

خانه > ایجاد خط تردد

ایجاد خط تردد

فیلترها

استان

انتخاب استان

شهرستان

انتخاب شهرستان

بخش

انتخاب بخش

روستا (اختیاری)

انتخاب روستا

بازنشانی فیلترها

خط تردد

پایانه مبدأ

انتخاب کنید

فاصله تقریبی (کیلومتر)

پایانه مقصد

انتخاب کنید

زمان تقریبی (دقیقه)

قیمت پایه (تومان)

ذخیره

انصراف

ایجاد خط تردد با راستی آزمایی هر مقدار و نمایش پیام های مناسب برای هر کاستی (ترجمه شده به فارسی)

5.2.5.3- صفحه ویرایش خط تردد

خانه > ویرایش خط تردد

فیلترها

استان

انتخاب استان

بخش

انتخاب بخش

روستا (مختیاری)

انتخاب روستا

بازنشانی فیلترها

خط تردد

پایانه مبدأ

خیابان ملک - گیلان - آستارا - مرکزی - آستارا

پایانه مقصد

خیابان فهمیده - فارس - سروستان - مرکزی - شوریجه

فاصله تقریبی (کیلومتر)

۱۴۶۵

زمان تقریبی (دقیقه)

۲۱۶

قیمت پایه (تومان)

۱۰۶۱۲۰۸

انصراف

ذخیره تغییرات

ویرایش خط تردد با راستی آزمایی هر مقدار و نمایش پیام های مناسب برای هر کاستی (ترجمه شده به فارسی) + فیلتر منطقه (مبدا و مقصد) جهت بهبود تجربه کاربری برای انتخاب پایانه

5.3- مدیریت سرویس‌های تردد

5.4- شبیه‌ساز اشغالی - Occupancy Simulator

5.4.1- هدف و ورودی‌ها

این سرویس یک دمو زمان‌بندی شده است که برای نمایش عملکرد موتور قیمت‌گذاری در شرایط متغیر اشغال طراحی شده است. ورودی: تمام رکوردهای جدول `transit_services`

5.4.2- مراحل پردازش

1. زمان‌بند `Laravel` هر دقیقه `OccupancySimulatorService::simulate()` را فراخوانی می‌کند.
2. سرویس تمام سرویس‌ها را بارگذاری می‌کند.
3. برای هر سرویس، `occupancy` به‌صورت تصادفی بین -0.3 و $+0.03$ تغییر می‌کند.
4. مقدار جدید در بازه $[0, 1]$ محدود می‌شود.
5. سرویس ذخیره شده و سپس `TransitPricingService::updatePrice()` فراخوانی می‌گردد.

5.4.3- فایل‌ها و کلاس‌های درگیر

- `app/Services/Demo/OccupancySimulatorService.php`
- `routes/console.php`

5.4.4- خروجی و نکات فنی

خروجی: تغییر لحظه‌ای قیمت سرویس‌ها در فهرست و افزایش مداوم رکوردهای تاریخچه.
نکات فنی: - این شبیه‌ساز بدون هیچ شرطی تمام سرویس‌ها را هر دقیقه پردازش می‌کند که با افزایش حجم داده، بار پایگاه‌داده را به‌صورت خطی افزایش می‌دهد. - در محیط `Production` باید محدودیت‌هایی برای اجرای آن در نظر گرفته شود.

5.5- آبخاری انتخاب مناطق جغرافیایی

5.5.1- هدف و ورودی‌ها

این قابلیت `Dropdown` های وابسته استان، شهرستان، بخش، آبادی و روستا را در فرم‌های پایانه و خط تردد فراهم می‌کند. ورودی: شناسه سطح والد (مثلاً `province_id` برای دریافت شهرستان‌ها)

5.5.2- مراحل پردازش

1. `Trait withRegionSelects` آرایه‌های محاسبه‌شده `ProvinceOptions` تا `VillageOptions` را تعریف می‌کند.
2. هنگام تغییر هر `Dropdown` (مثلاً استان)، متد `updated` مربوطه فرزندان را `reset` می‌کند.
3. `RegionsService::getSelectOptionsByRegion($regionKey, $parentId)` گزینه‌ها را از پایگاه‌داده استخراج می‌کند.
4. `RegRegionTrait::selectOptionsFor()` خروجی را برای `Dropdown` آماده می‌سازد.

5.5.3- فایل ها و کلاس های درگیر

- `app/Traits/withRegionSelects.php`
- `app/Services/RegionsService.php`
- `app/Models/IranRegion/RegRegionTrait.php`

5.5.4- خروجی و نکات فنی

خروجی: Dropdown های آبخاری با بارگذاری پویا بر اساس انتخاب کاربر.

نکات فنی: - رابطه `Settlement::parent` تعریف شده اما در هیچ بخشی از برنامه استفاده نشده است و آرگومان های آن نیاز به بررسی دارند.

فصل ششم: موتور قیمت گذاری پویا

6.1- هدف قابلیت

این موتور قلب تپنده پروژه است. هدف آن، محاسبه قیمت نهایی هر سرویس تردد بر اساس ترکیبی از عوامل متغیر و ثابت است. این رویکرد امکان قیمت‌گذاری هوشمندانه بر اساس شرایط لحظه‌ای سفر را فراهم می‌کند.

6.2- ورودی‌ها

ورودی‌های موتور از Model TransitService و رابطه transitLine استخراج می‌شوند:

- base_price: قیمت پایه خط تردد
- estimated_distance_km: مسافت تخمینی خط
- estimated_duration_min: مدت زمان تخمینی خط
- departure_time: زمان حرکت سرویس
- occupancy: نسبت اشغال صندلی‌ها
- is_vip: وضعیت VIP

6.3- مراحل پردازش

1. TransitPricingService::updatePrice(\$service) سرویس و خط مربوطه را بارگذاری می‌کند.
2. قیمت پایه از خط به سرویس همگام‌سازی می‌شود.
3. متد calculatePrice(\$service) شش ضریب را محاسبه می‌کند.
4. قیمت نهایی با ضرب تمام ضرایب در قیمت پایه به دست می‌آید.
5. نتیجه round شده و حداقل مقدار 1 اعمال می‌شود.
6. یک رکورد تاریخچه با تمام ضرایب، قیمت قبلی، مقدار تغییر و درصد تغییر ثبت می‌شود.
7. قیمت جدید در computed_price سرویس ذخیره می‌گردد.

6.4- منطق اصلی

فرمول محاسبه قیمت به شرح زیر است:

$$\text{price} = \text{base_price} \times \text{distance_factor} \times \text{duration_factor} \times \text{time_factor} \times \text{occupancy_factor} \times \text{distance_accuracy_factor} \times \text{vip_factor}$$

ضرایب:

- **distance_factor**: بر اساس `estimated_distance_km` با اعمال یک `tolerance` نرخ ثابت محاسبه می‌شود.
 - **duration_factor**: مدت زمان بسیار کوتاه یا بسیار طولانی را جریمه می‌کند.
 - **time_factor**: بر اساس دقیقه‌های باقی‌مانده تا `departure_time` محاسبه می‌شود؛ هرچه زمان بیشتر باشد، قیمت کمتر.
 - **occupancy_factor**: با نزدیک شدن `occupancy` به 1، به صورت نمایی افزایش می‌یابد.
 - **distance_accuracy_factor**: در صورتی که مسافت واقعی کوتاه‌تر از تخمین باشد، قیمت را کاهش می‌دهد.
 - **vip_factor**: در صورت فعال بودن `is_vip` مقدار 1.15 و در غیر این صورت 1 است.
- Edge Handling**: - محافظت در برابر تقسیم بر صفر - `round()` به نزدیک‌ترین عدد صحیح - `max(price, 1)` برای جلوگیری از قیمت صفر - محدودسازی ضرایب در بازه‌های منطقی

6.5- فایل‌ها و کلاس‌های درگیر

- `app/Services/TransitPricingService.php`
- `app/Models/TransitService.php`
- `app/Models/TransitServicePriceHistory.php`
- `app/Observers/TransitLineObserver.php`

6.6- تعامل با پایگاه داده

موتور در هر اجرا دو عملیات نوشتنی انجام می دهد:

1. به روزرسانی `computed_price` و `base_price` در جدول `transit_services`
2. درج یک رکورد جدید در جدول `transit_service_price_histories`

6.7- خروجی

خروجی: مقدار `computed_price` به روزرسانی شده در `Model` سرویس و یک رکورد حسابرسی کامل در تاریخچه.

6.8- نکات فنی مهم

- متد `calculatePrice()` خالص (pure) است و عوارض جانبی ندارد؛ بنابراین قابل تست واحد است.
- متد `updatePrice()` عوارض جانبی دارد (ذخیره در پایگاه داده) و در `Observer` و شبیه ساز فراخوانی می شود.
- عدم استفاده از `Transaction` می تواند در شرایط خطا منجر به ناسازگاری بین قیمت فعلی و آخرین رکورد تاریخچه شود.

فصل هفتم: بررسی فایل‌های مهم پروژه

7.1- لایه Service

- `app/Class/Service.php`
 - این کلاس `abstract` پایه تمام سرویس‌ها است. متدهای اصلی آن عبارتند از: `paginate()`: صفحه‌بندی نتایج - `applySearch()`: اعمال عبارت جستجو - `applySort()`: اعمال مرتب‌سازی - `applyFilters()`: اعمال فیلترهای سفارشی - `create()` / `update`: ثبت و ویرایش - `delete()` / `deleteMany`: حذف تک‌رکوردی و گروهی
- `app/Services/TransitPricingService.php`
 - موتور قیمت‌گذاری پویا. متدهای کلیدی: `updatePrice($service)`: محاسبه و ذخیره قیمت + تاریخچه - `calculatePrice($service)`: محاسبه خالص بدون عوارض جانبی
- `app/Services/TerminalService.php`
 - سرویس پایانه‌ها با محافظت حذف و فیلتر منطقه‌ای.
- `app/Services/TransitLineService.php`
 - سرویس خطوط تردد با فیلتر مبدأ/مقصد و محافظت حذف.

7.2- لایه Model

- `app/Models/Terminal.php`
 - مدل پایانه با `address` Accessor () برای ساخت آدرس فارسی.
- `app/Models/TransitLine.php`
 - مدل خط تردد با روابط `originTerminal`، `destinationTerminal` و `services`.
- `app/Models/TransitService.php`
 - مدل سرویس با `cast`های `is_vip` و `occupancy`.
- `app/Models/TransitServicePriceHistory.php`
 - مدل تاریخچه با `cast` تاریخ `Jalali`.

7.3- لایه View و Livewire

- `resources/views/layouts/app.blade.php`
Layout اصلی با `<html dir="rtl" lang="fa">` و بارگذاری فونت `IRANSans`
- `resources/views/pages/⚡dashboard.blade.php`
داشبورد با کارت‌های میانبر به صفحات اصلی.
- `resources/views/pages/terminals/⚡index/index.php`
Component ناشناس فهرست پایانه‌ها با ترکیب Trait های `withRegionSelects`، `withModal`، `withDatatable` و `withTableDelete`
- `app/Livewire/Terminals/Modals/Delete.php`
Component مودال تأیید حذف پایانه.

7.4- لایه پیکربندی و Route

- routes/web.php

تمام 12 مسیر برنامه با Route::livewire () تعریف شده‌اند. مثال:

```
Route::prefix('terminals')->name('terminals.')->group(function() {  
    Route::livewire('/', 'pages::terminals.index')->name('index');  
    Route::livewire('/create', 'pages::terminals.form')->name('create');  
    Route::livewire('/{terminal}/edit', 'pages::terminals.form')->name('edit');  
});
```

- config/livewire.php

پیکربندی namespace صفحات (pages::)، Layout پیش‌فرض (layouts::app) و تنظیمات SFC.

- routes/console.php

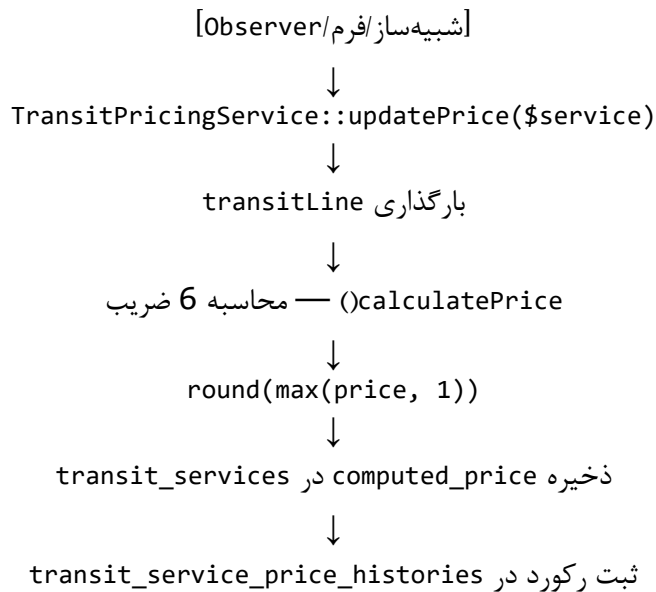
زمان‌بندی شبیه‌ساز اشغالی:

```
Schedule::call(function() {  
    app(OccupancySimulatorService::class)->simulate();  
}-({everyMinute;()
```

فصل هشتم: جریان‌های اصلی سیستم

8.1- جریان محاسبه مجدد قیمت پویا

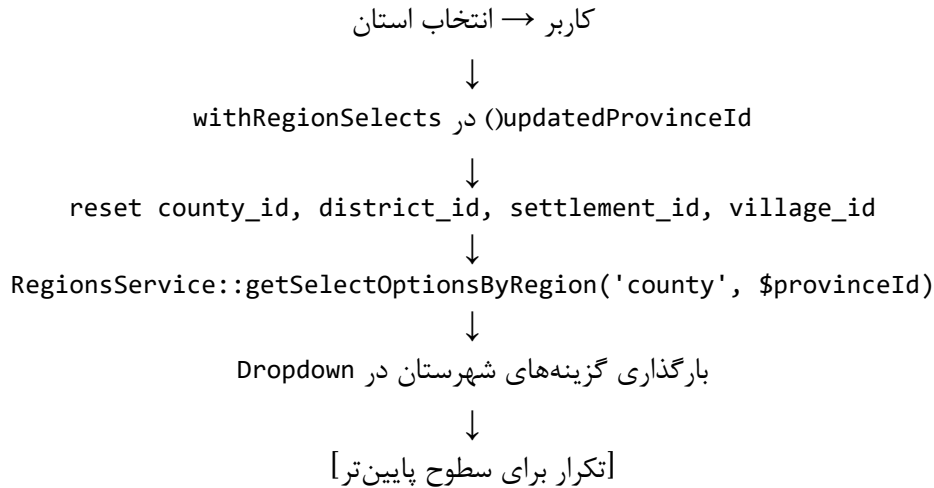
این جریان در سه سناریو اجرا می‌شود: ایجاد/ویرایش سرویس، ویرایش خط تردد، و اجرای شبیه‌ساز.



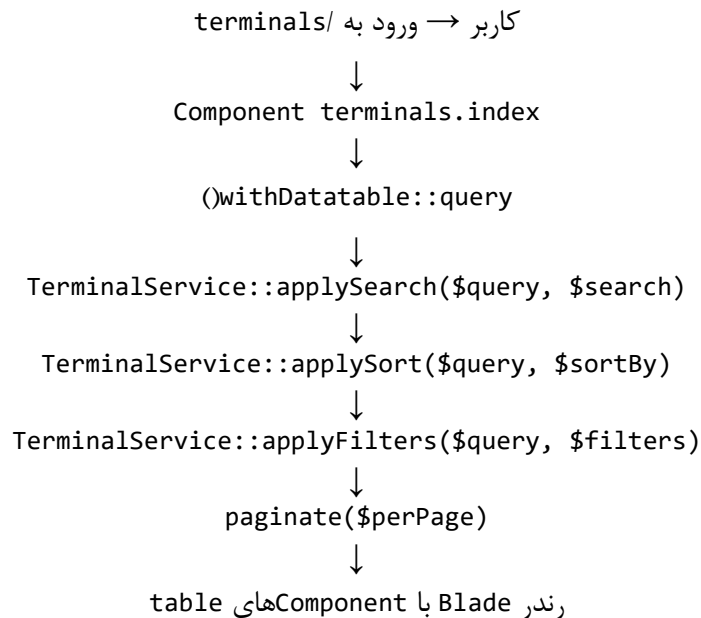
8.2- جریان حذف با محافظت و مودال



8.3- جریان آبخاری انتخاب منطقه



8.4- جریان نمایش داده‌های جدول (Datatable)



نگاهی به آینده

TransitApp در نسخه فعلی یک هسته مدیریتی قوی برای دامنه حمل و نقل بین شهری فراهم کرده است. با این حال، برای تبدیل شدن به یک محصول عملیاتی، مسیرهای زیر پیشنهاد می شود:

- سامانه رزرواسیون: اتصال سرویس ها به سیستم فروش بلیط آنلاین با کاهش ظرفیت واقعی
- پنل کاربری: ایجاد پنل برای مسافران جهت مشاهده سرویس ها و قیمت ها
- API عمومی: ارائه API RESTful برای اپلیکیشن های موبایل و سامانه های ثانویه
- گزارش های تحلیلی: داشبوردهای تحلیلی برای مدیران جهت بررسی پرتددترین خطوط و پایانه ها
- اعلان ها: سیستم اعلان برای تغییرات قیمت یا llen شدن ظرفیت سرویس
- یکپارچه سازی نقشه: نمایش مسیر خطوط تردد روی نقشه و محاسبه خودکار مسافت از طریق API نقشه
- پشتیبانی از سایر وسایل نقلیه مثل اتوبوس و قطار
- اتصال به سیستم های امداد خودرو و خدمات اضطراری
- ردیابی gps وسایل نقلیه
- بیمه خودکار برای سرنشین ها و مسافران

منابع

- Laravel Documentation: <https://laravel.com/docs>
- Livewire Documentation: <https://livewire.laravel.com/docs>
- Tailwind CSS Documentation: <https://tailwindcss.com/docs>
- MySQL Documentation: <https://dev.mysql.com/doc/>
- morilog/jalali (Jalali Date): <https://github.com/morilog/jalali>
- wire-elements/modal: <https://github.com/wire-elements/modal>
- Laravel Tailwind CSS Admin Dashboard Template: <https://tailadmin.com/laravel>